

Review of the Proposed Pydantic Predicate Implementation

Executive Summary

The proposed implementation is a strong Phase 1 design.

It achieves an important goal:

```
User-facing object contracts
      ↓
Verifier-level predicates
      ↓
Existing flat-state backend
```

without requiring any changes to:

- Coq definitions
- WP generation
- SMT encoding
- The theorem generation pipeline

This is exactly the kind of incremental evolution that should be preferred in Axiomander.

However, there are several semantic issues that should be addressed before implementation proceeds.

The largest issue is that the proposed meaning of:

```
pydantic_model(obj, ModelType)
```

does not match what the implementation actually proves.

The implementation proves a shape property, not full Pydantic validity.

Overall Assessment

What Is Good

The proposal correctly recognizes that:

```
account.balance
```

is the abstraction users want to reason about,

while:

```
account_balance
```

is merely an implementation detail of the verifier.

The addition of:

```
field(account, "balance", b)
```

creates a useful abstraction layer.

This improves:

- readability
- maintainability
- future extensibility

without requiring any changes to the backend proof engine.

This is a good architectural direction.

Major Concern: Predicate Semantics

Current Proposal

The proposal defines:

```
pydantic_model(account, Account)
```

as:

```
account is a validated instance of Account
```

but compiles it into:

```
isVZ (s "account_balance"%string) = true /  
isVZ (s "account_overdraft_limit"%string) = true
```

This proves only that:

- expected fields exist
- fields have expected primitive verifier types

It does not prove:

- Pydantic validators executed
- Field constraints satisfied
- Model invariants satisfied
- Runtime validation succeeded

Therefore the logical meaning is stronger than the implementation.

This should be corrected.

Recommendation: Rename the Predicate

Instead of:

```
pydantic_model(account, Account)
```

introduce:

```
pydantic_shape(account, Account)
```

or:

```
model_shape(account, Account)
```

The logical meaning becomes:

```
account has the declared shape of Account
```

which exactly matches the implementation.

Reserve a Future Predicate

A later phase may introduce:

```
pydantic_valid(account, Account)
```

which represents:

```
all declared Pydantic validation semantics hold
```

including:

- Field(...)
- validators
- model validators
- custom constraints

This future predicate can be lowered differently.

The naming distinction should be established now.

Missing Model Should Be an Error

Current Proposal

If the linter cannot find:

```
Account
```

the proposal suggests expanding to:

```
True
```

This is unsafe.

Example:

```
requires:  
    pydantic_model(account, Account)
```

becomes:

```
True
```

if Account cannot be found.

The user believes a constraint exists.

The verifier silently removes it.

This is unsound.

Recommendation

Failure to locate a model should produce:

```
Verification Error:  
Unable to locate Pydantic model Account
```

or at minimum:

```
Verification Warning:  
Model shape could not be expanded
```

A contract predicate should never disappear silently.

Field Predicate Review

Proposed Form

```
field(account, "balance", b)
```

This is a good abstraction.

The lowering:

```
asZ (s "account_balance"%string) = b
```

fits naturally into the existing architecture.

No concerns here.

Missing Field Handling

The proposal already includes a negative test for:

```
field(account, "not_real", x)
```

This should become a required semantic check.

Recommendation

When model information is available:

```
field(account, "balance", b)
```

must verify that:

```
balance ∈ fields(Account)
```

Otherwise:

```
Verification Error:  
Unknown field balance for Account
```

should be produced.

Relationship to Existing Pydantic Constraints

Currently Axiomander already extracts:

```
Field(ge=...)  
Field(le=...)
```

and converts them into verifier preconditions.

This is important because it means the verifier already understands some Pydantic semantics.

The new shape predicates should build on this mechanism rather than replace it.

The progression should be:

```
Phase 1  
  Shape existence  
  
Phase 2  
  Shape existence  
  +  
  Field constraints  
  
Phase 3  
  Shape existence  
  +  
  Field constraints  
  +  
  Validators  
  
Phase 4  
  Shape existence  
  +  
  Full model validity
```

Deferred Logical Data Variables

The proposal intentionally rejects:

```
pydantic_model(user, User, U)
```

where:

```
U
```

is a logical map representing the model.

This is the correct decision.

Supporting this would require:

- map types
- record types
- projection operators
- update operators
- SMT encoding
- Coq encoding

This would significantly increase complexity.

Deferring this feature is recommended.

Architectural Observation

The proposal is strongest when viewed as:

```
Pydantic Frontend
  ↓
Shape Predicate Layer
  ↓
Flat State Representation
  ↓
Existing Proof Infrastructure
```

and not as:

```
Pydantic Frontend
  ↓
Direct Verification Semantics
```


The distinction is important.

The verifier should reason about shape predicates.

The shape predicates may originate from Pydantic.

They are not themselves Pydantic.

Recommended Revised API

Instead of:

```
pydantic_model(account, Account)
```

use:

```
pydantic_shape(account, Account)
```

and:

```
field(account, "balance", b)
```

Example:

```
requires:
    pydantic_shape(account, Account)
    field(account, "balance", b)
    amount >= 0
    b >= amount

ensures:
    field(account, "balance", b - amount)
```

This precisely matches the implementation.

Final Verdict

The proposal is a strong Phase 1 implementation and should largely proceed.

However, before implementation:

1. Rename `pydantic_model` to `pydantic_shape` (or equivalent).
2. Make missing model lookup a verification error.
3. Make unknown field references a verification error.
4. Reserve `pydantic_valid` for future full-validation semantics.
5. Continue treating the predicates as shape abstractions over the existing flat-state encoding.

With these changes the design becomes both practical and semantically honest, while preserving a clean migration path toward richer object, database, and ownership reasoning in future versions of Axiomander.